

# Chapter 6: Object-Relational Mappers

Gregory S. DeLozier, PhD

The database layer gives the application operations such as `get_pet` and `update_pet`. Until now, we have mostly implemented those operations by writing Structured Query Language (SQL) statements. This time we will use Python classes and expressions to describe the same kinds of work.

An **object-relational mapper (ORM)** connects objects in a program with records in a relational database. (1) We will use Peewee. It lets us define a model class and then create or query records through that class. (2)

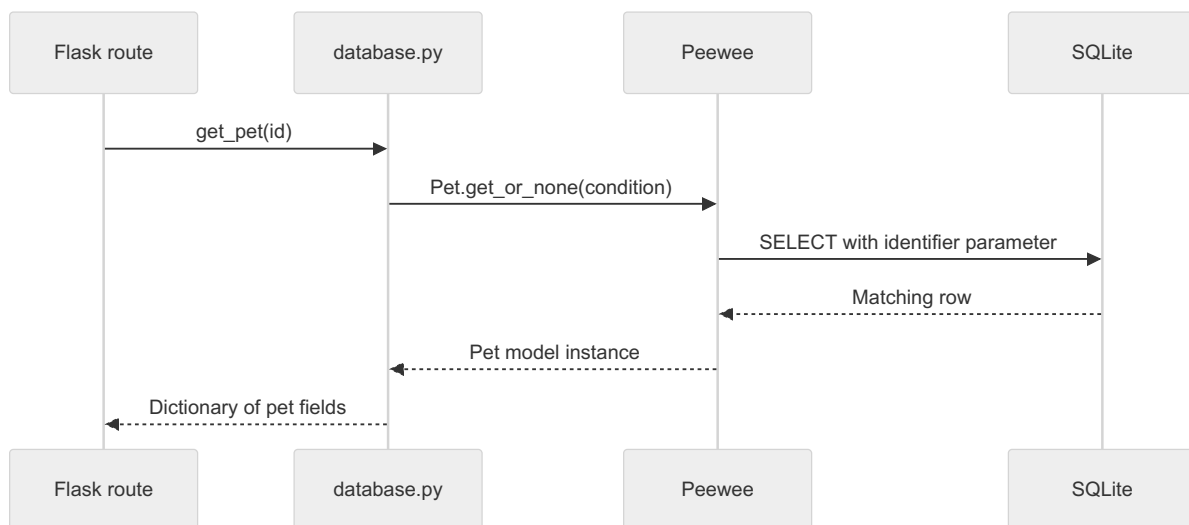
This example focuses on one pet table, with name, type, age, and food. That gives us room to follow the mapping before working with owner relationships in the ORM-with-constraints example. Use the files in `chapter-06-object-relational-mappers/code`; this app has its own database and forms.

## The Application Still Calls the Database Layer

The route asks for a record in the familiar way:

```
data = database.get_pet(id)
```

It receives a dictionary or `None`. The route does not need to know whether the function used a cursor or a Peewee query. We keep that interface while changing its implementation.



The diagram follows a successful lookup. A missing record produces `None` instead. SQLite still stores the table. Peewee takes over the work of generating SQL and turning returned rows into model instances.

There are two abstraction boundaries here. Our functions describe operations the application needs. Peewee provides tools for implementing those operations. Keeping our functions means the route can continue working with the same arguments and results.

## Running the Example

From a working copy of the code directory:

```
python3 -m pip install -r requirements.txt
flask run
```

Open:

```
http://127.0.0.1:5000/list
```

The requirements file pins the versions used to test this example: Peewee 4.0.0 and Flask 3.1.3. The application prepares `pets.db` when it starts. The separate command below prepares the database without starting Flask:

```
python3 setup_database.py
```

`session.txt` contains an earlier interactive exploration of Peewee, including related models in `people.db`. Our application uses `pets.db` and the `Pet` model in `database.py`. The chapter's examples follow that module.

## Models, Fields, and Records

Here is the model definition:

```
class Pet(BaseModel):
    # Class attributes describe columns. Peewee supplies an integer id field.
    name = TextField(null=False)
    type = TextField(null=False)
    age = IntegerField(default=0)
    food = TextField(null=True)
```

The class describes the table. A field describes a column. An instance represents a particular row. Peewee supplies an integer primary key named `id` when a model does not declare one. (3)

| Python expression     | Meaning in this example              |
|-----------------------|--------------------------------------|
| <code>Pet</code>      | Model for the pet table              |
| <code>Pet.name</code> | Field used when constructing a query |
| <code>pet.name</code> | Name value on a particular instance  |
| <code>pet.id</code>   | Identifier of that pet               |

Capitalization matters in this discussion: `Pet` is the class, while `pet` is a variable holding one instance. `Pet.name` helps describe a database operation. `pet.name` gives us a value to display or change.

The field options belong to the model definition. `null=False` disallows a missing database value. `food` permits one. `default=0` provides a Python-side default for a new model instance; it does not add a SQL `DEFAULT` clause to the table. (3)

## Selecting the Database

The module creates a Peewee database object before choosing a filename:

```
db = SqliteDatabase(None)

class BaseModel(Model):
    class Meta:
        database = db
```

`Meta` contains model configuration. Here it associates models derived from `BaseModel` with `db`. We create the `Pet` table, not a separate `BaseModel` table. (3)

The module name `database` and the object `db` have different jobs. `database.get_pet()` is our application's function. `db` is the Peewee object used inside that module to manage database access.

Initialization supplies the filename and prepares the table:

```
def initialize(database_file):
    close_connection()
    db.init(database_file)
    db.connect(reuse_if_open=True)
    db.create_tables([Pet])
    # The remaining setup code adds food to an older table when needed.
```

Peewee supports this deferred initialization so the same models can use a filename selected at runtime. (4) Our app selects `pets.db`; tests select temporary files.

Creating a missing table and changing an existing table are separate operations. (2)

To **migrate** a database here means to bring its existing structure, or **schema**, from an earlier version to the version the application now expects. A schema migration might add a column, rename one, or change a constraint. (5) The application has changed, but we still need the records people have already entered.

Our example started with a `pet` table that had no `food` column. The migration adds that column to the existing table and keeps the `pet` records. Those older records initially have `NULL` for `food`, because no `food` value was stored for them. The database stays in the same file; this migration changes its structure rather than moving it to another server.

The code inspects the existing columns and adds the nullable field only when necessary:

```
columns = {column.name for column in db.get_columns("pet")}
if "food" not in columns:
    migrate(SqliteMigrator(db).add_column(
        "pet", "food", TextField(null=True)
    ))
```

SqliteMigrator and migrate come from Peewee's `playhouse.migrate` module. This adds the column while preserving the existing records. (5)

## Keeping the Function Interface

The route-facing operations remain small:

| Function                          | Return value         |
|-----------------------------------|----------------------|
| <code>get_pets()</code>           | List of dictionaries |
| <code>get_pet(id)</code>          | Dictionary or None   |
| <code>create_pet(data)</code>     | New pet identifier   |
| <code>update_pet(id, data)</code> | None                 |
| <code>delete_pet(id)</code>       | None                 |

Inside the module, Peewee returns model instances. We convert those to dictionaries at the boundary:

```
def pet_to_dict(pet):
    return {
        "id": pet.id, "name": pet.name, "type": pet.type,
        "age": pet.age, "food": pet.food,
    }
```

This conversion is our design choice. The templates already use expressions such as `pet['food']`, so we continue supplying that representation. A caller does not acquire a dependency on Peewee merely by asking the database layer for a record.

## Reading Pets

The list function builds a query and then walks through its results:

```
def get_pets():
    query = Pet.select().order_by(Pet.name, Pet.id)
    return [pet_to_dict(pet) for pet in query]
```

`select()` constructs a query. Iteration executes it and yields model instances. (6) The function sorts by name and then by identifier, giving equal names a consistent order. Each instance becomes a dictionary before the list leaves the database layer.

For a single pet:

```
def get_pet(id):
    pet = Pet.get_or_none(Pet.id == int(id))
    if pet is None:
        return None
    return pet_to_dict(pet)
```

The comparison `Pet.id == int(id)` builds a SQL condition. Peewee overloads the field's comparison operation to create an expression; it is not comparing the identifier of an already loaded pet. `get_or_none()` executes the lookup and returns `None` when no row matches. (6)

The Flask edit route checks that result. A dictionary supplies the form values. `None` produces a not-found response.

## Creating a Pet

The form submits a dictionary of strings. Our `_pet_values` helper strips the name and type, checks that they are present, and converts age. It keeps the existing example's rule of using zero when age cannot be converted to an integer.

The write function then calls Peewee:

```
def create_pet(data):
    pet = Pet.create(**_pet_values(data))
    return pet.id
```

The `**` expands the dictionary into keyword arguments. With Casey's form values, the call is equivalent to:

```
pet = Pet.create(
    name="Casey", type="dog", age=9, food="kibble"
)
```

`create()` inserts the record and returns the saved instance. (7) Our wrapper returns only the assigned identifier. The route then redirects to the list, whose next query retrieves the saved pet.

Compare the responsibilities with the earlier SQL version. The helper prepares the values. The model supplies the field definitions. Peewee builds the insert. Our function still decides what its caller receives.

## Updating and Deleting

The application saves an edited pet through this function:

```
def update_pet(id, data):
    Pet.update(**_pet_values(data)).where(
        Pet.id == int(id)
    ).execute()
```

`update()` describes the new values, `where()` selects the record, and `execute()` runs the statement. Deletion follows the same pattern. (7)

```
def delete_pet(id):
    Pet.delete().where(Pet.id == int(id)).execute()
```

Suppose Casey has identifier 10. Changing the form's food value from kibble to chicken produces:

```
database.update_pet(10, {
    "name": "Casey", "type": "dog", "age": "9",
    "food": "chicken"
})
```

The identifier passed separately selects Casey. The dictionary supplies the replacement values. Reading pet 10 again returns chicken as the stored food.

The condition deserves attention. It states which rows an update or delete affects. Our functions use the primary key because each request concerns one pet.

## An Object and Its Saved Record

Peewee also allows us to construct an object and save it explicitly. (2) Try this in an interactive Python session from the code directory, using a separate practice file:

```
import database
from database import Pet

database.initialize("practice.db")
pet = Pet(name="Casey", type="dog", age=9, food="kibble")
pet.save()

pet.food = "chicken"
print(database.get_pet(pet.id)["food"]) # kibble

pet.save()
print(database.get_pet(pet.id)["food"]) # chicken

database.close_connection()
```

The two reads make the distinction visible. Attribute assignment changes the object. `save()` writes the change. Our web layer uses `Pet.update(...).execute()` instead, so it can submit the edited values directly without first loading a model instance. Both operations have a clear point where they write to the database. (7)

## Where Is the Commit?

Our earlier functions called `connection.commit()`. The Peewee functions above use its normal autocommit behavior: outside an explicit transaction, each successful write completes independently. (8)

When several operations must succeed together, Peewee provides `atomic()`:

```
with database.db.atomic():
    database.create_pet({
        "name": "Casey", "type": "dog", "age": 9, "food": "kibble"
    })
    database.create_pet({
        "name": "Heidi", "type": "cat", "age": 15, "food": "tuna"
    })
```

The block commits on successful completion. An exception escaping the block causes a rollback. (8) This gives us a way to group the same database-layer functions without rewriting their queries. The test suite includes a deliberately cancelled operation and confirms that its inserted pet does not remain.

## Looking at the SQL

The ORM still sends SQL to SQLite. Peewee can show the statement and parameters for a query through `sql()`. (6) In a Python session:

```
import database
from database import Pet

database.initialize("pets.db")
query = Pet.select().where(Pet.type == "cat")
statement, parameters = query.sql()
print(statement)
print(parameters)
database.close_connection()
```

Find the selected columns and the condition in the output. The parameter list contains the value `cat`. Compare that with the handwritten query we would use for the same request:

```
SELECT id, name, type, age, food
FROM pet
WHERE type = ?;
```

The generated statement may include quoted names and a table alias. It performs the same selection. Reading both versions connects the Python expression to the work requested from the database.

## Tests and Practice

The short embedded tests remain available:

```
python3 database.py
```

They recreate `test_pets.db` and exercise the database functions. The additional tests use temporary files:

```
python3 -m unittest -v
```

Those tests check persisted values after reconnecting. They also check explicit saving, rollback, and the addition of food to an older table. An independent `sqlite3` query confirms that the ORM writes the expected value into the real pet table.

For practice, add `get_pets_by_type(pet_type)` to the database layer. Return a list of dictionaries in name order, just as `get_pets()` does. Test a type with several pets and a type with no matches. The second result should be an empty list.

Then add a food filter. Keep Peewee expressions in `database.py`, and let a route supply the chosen value. Inspect the generated SQL and identify the parameter holding that value. Explain which files changed when adding the operation.

## References

All sources accessed September 22, 2026.

1. Wikipedia contributors. “Object-relational mapping.” Wikipedia. [https://en.wikipedia.org/wiki/Object%E2%80%93relational\\_mapping](https://en.wikipedia.org/wiki/Object%E2%80%93relational_mapping)
2. Charles Leifer. “Quickstart.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/quickstart.html>
3. Charles Leifer. “Models and Fields.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/models.html>
4. Charles Leifer. “Database.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/database.html>
5. Charles Leifer. “Database Tooling: Schema Migrations.” Peewee documentation. [https://docs.peewee-orm.com/en/latest/peewee/db\\_tools.html#schema-migrations](https://docs.peewee-orm.com/en/latest/peewee/db_tools.html#schema-migrations)
6. Charles Leifer. “Querying.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/querying.html>
7. Charles Leifer. “Writing Data.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/writing.html>
8. Charles Leifer. “Transactions.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/transactions.html>



## Questions for Thought

1. What would become easier or harder if `get_pet()` returned a Peewee model instance instead of a dictionary? Consider both the template and a future replacement for Peewee.
2. A programmer adds a field to `Pet` but keeps an existing database file. Why should table creation and schema changes be treated as different operations?
3. Suppose two pet records must be created together. Where would you put the transaction boundary, and how would you test failure after the first insert?
4. When would inspecting the generated SQL help you understand or improve an ORM query? What information does the Python expression leave implicit?